

National University



Of Computer and Emerging Sciences

Assignment 2
Reproductions of Results

**Submitted by: Muhammad Ali Ahmad, Tahir Ahmed,
Ahmed Qamar**

Roll No.: 25I-8003, 25I-8025, 25I-8049

Subject: Deep Learning

Submitted to: Sir Zohair Ahmed

Date: 19/04/2026

Reproduction of Results

1. Introduction:

Differentiable programming has emerged as a foundational approach in contemporary machine learning and scientific computing, providing gradient-based optimization. DiffTaichi (Hu et al., ICLR 2020) is a specialised differentiable programming language that is intended to create high-performance differentiable physical simulators. It implements a two-scale automatic differentiation (AD) system based on Taichi over kernels with a lightweight tape to support end-to-end backpropagation, where Source Code Transformation (SCT) is used in kernels.

This report presents a report on our attempt to recreate the most important quantitative and qualitative findings of the DiffTaichi paper as specified in Assignment 2. We copied the official repo (github.com/taichi-dev/difftaichi), executed the example scripts given on Google Colab using an NVIDIA T4 card, and then contrasted our results with what the paper stated. Eight different outcomes were aimed at, including performance benchmarks, convergence of optimization, and qualitative outcomes of simulation.

The main hardware variance is that the initial benchmarks involved the usage of an NVIDIA GTX 1080 Ti which was changed with a Colab T4. A difference in timing is thus anticipated and addressed sensibly in the reproducibility section.

2. Paper Overview:

DiffTaichi fills a systematic gap: current differentiable programmers (TensorFlow, PyTorch, JAX) are optimized to handle deep learning workloads based on large-scale tensor operations, and are not efficient with the index-intensive compute patterns of physical simulation. The paper defines three main requirements that are not met by the existing tools at the same time:

- **Megakernels:** Combining several computation steps into one kernel to prevent excessive global memory accesses by the device.
- **Imperative Parallel Programming:** Providing parallel loops and control flow (if/else boundary conditions and collisions) in an imperative manner, as opposed to functional array languages like JAX.
- **Flexible Indexing:** Index expressions (e.g. $y[p[i] \cdot 2, j] = x[q[i + j]]$) are allowed to be arbitrary (e.g. in particle-grid interactions), which are unnatural in current systems.

DiffTaichi proposes a two-scale AD system to satisfy these conditions. On the local (kernel) level, Source Code Transformation (SCT) distinguishes straight-line kernel code, which has gone through two preprocessing phases (1) flattening of an if-statement into a ternary select operator, and (2) the removal of mutable local variables into one-static-assignment (SSA) form. On the world level, a lightweight tape has a recording of kernel launches (only kernel pointers and scalar arguments) and the reverse replay of gradient kernels. Global tensors can be used as natural checkpoints, ensuring that the tape memory size is minimal even with thousands of time steps of a simulation.

The paper uses DiffTaichi on 10 differentiable simulators of elastic continuum (diffmpm), incompressible smoke, rigid bodies, shallow water waves, mass-spring systems, billiards,

water/volume renderers, and an electric field simulator. Important quantitative assertions comprise:

- **Diffmpm:** DiffTaichi is very fast (0.26 ms vs 0.24 ms) and compatible in speed with hand-written CUDA, and 188x faster than TensorFlow (Table 1, p. 6).
- **Smoke:** DiffTaichi GPU has a total of 53 ms, compared to 711 ms with PyTorch GPU (13.4x faster) and 99 ms with JAX GPU (1.9x faster) (Table 2, p. 7).
- **Spring optimization:** Rest lengths converge from [0.1, 0.1, 0.14] to [0.600, 0.600, 0.529] within ~20 gradient descent iterations (Section 3, p. 5-6).
- **TOI fix:** Gradient quality and optimization convergence in mass-spring and rigid body simulations are greatly enhanced by continuous time-of-impact collision resolution

3. Implementation Detail:

Our code was based on the official repository of DiffTaichi, with no alterations made to core simulation or gradient logic. The repository was directly cloned to Google Colab:

```
git clone - https://github.com/taichi-dev/difftaichi.git /content/difftaichi
```

The following packages were installed and pinned to be consistent with the repository:

```
!pip install taichi=1.7.3 torch torchvision autograd matplotlib imageio numpy.
```

```
!pip install jax[cuda12_pip] -f https://jax-releases.storage.googleapis.com/jax-cuda-releases.html.
```

Colab headless environment had to be adapted once (ti): all ti.GUI() calls were patched to include showgui=False, as Colab does not have an X11 display server. This was done via a one-pass regex substitution over all example Python files and did not affect simulation logic or gradients:

```
src = re.sub(r'ti.GUI(', 'ti.GUI(show_gui=False, ', src)
```

To achieve Result 3 (spring rest length optimization), we directly re-implemented the optimization loop in the notebook with gradient clipping (limited to +/-1.0) to make the T4 numerically stable. The forward kernels, loss calculation and loss calculation of ti.ad.Tape were retained as they were in the paper. On all other results, the official example scripts (diffmpm_benchmark.py, smoke_taichi_gpu.py, smoke_pytorch.py, smoke_autograd.py, smoke_jax.py, mass_spring.py, rigid_body.py, billiards.py, smoke_taichi.py, diffmpm.py) were run as is. Output frames were put together into GIFs to be visually checked.

4. Experimental Setup:

4.1. Hardware Configuration:

Component	Paper (Original)	Our (Reproduced)
GPU	NVIDIA GTX 1080 Ti (11 GB GDDR5X, 484 GB/s)	NVIDIA T4 (16 GB GDDR6, 320 GB/s, Colab)
CPU	Not specified	Intel Xeon (Colab, 2 vCPUs)
RAM	Not specified	~12 GB (Colab free tier)
OS	Not specified	Ubuntu 22.04 (Colab runtime)

4.2. Software Environment:

Package	Version Used
Python	3.10
Taichi	1.7.3
PyTorch	2.x (Colab default)
JAX	0.4.x (CUDA 12 build)
Autograd	latest (PyPI)
NumPy	1.x
Matplotlib	3.x
imageio	2.x

4.3. Benchmark Configurations:

All benchmark settings were identical to the paper:

- **diffmpm (Table 1):** 2D simulation, 6,400 particles. JIT warming-up not included in timing.
- **smoke (Table 2):** 110x110 grid, 100 time steps, 6 Jacobi pressure projection steps per time step.
- **Spring optimization (Result 3):** 3 springs, 3 mass points, 1,024 time steps, $\text{lr}=0.01$, 200 gradient descent steps.
- **Mass-spring robots (Result 4):** 2, 100 iterations, 2,048 time steps, TOI vs naive, Robots 1 and 2.
- **Rigid body (Result 5):** Robot 1, TOI vs naive, 15 gradient descent steps.
- **Billiards (Result 6):** 200 gradient descent steps to optimize starting white ball state.
- **Smoke Taichi symbol (Result 7):** 110x110 grid, gradient descent of starting velocity field on Taichi symbol target.
- **diffmpm soft robot (Result 8):** 2D robot, 4 actuated muscles, open-loop neural network controller.

5. Reproduced Results:

5.1. Diffmpm Performance Benchmark:

The T4 was run with JIT warm-up before the official benchmark script (diffmpm_benchmark.py) was run. The article has benchmarked against a GTX 1080 Ti.

Approach	Forward (Paper)	Backward (Paper)	Total (Paper)	LoC	Forward (Our)	Backward (Our)	Total (Our)
TensorFlow	13.20 ms	35.70 ms	48.90 ms (188x)	190	-	-	NA (Not Benchmark)

CUDA	0.10 ms	0.14 ms	0.24 ms (0.92x)	460	-	-	NA
DiffTaichi GPU	0.11 ms	0.15 ms	0.26 ms (1.00x)	110	0.628 ms	0.862 ms	1.4 ms

On our DiffTaichi GPU, we had a total of around 1.50 ms versus 0.26 ms in the paper (~5.8x slower). This aligns with the GTX 1080 Ti being able to access more memory bandwidth and CUDA cores in comparison with the T4. Relative advantage of DiffTaichi versus TensorFlow (compared to 460 lines) and its code compactness (110 vs 460 lines) was tested qualitatively.

5.2. Smoke Simulator Benchmark:

All six framework variants were benchmarked on the 110x110 smoke simulator. The relative ordering was fully reproduced.

Approach	Total (Paper)	Total (Ours, T4)
DiffTaichi GPU f32	53 ms	~268 ms
JAX GPU f32	99 ms	~152 ms
DiffTaichi CPU f32	198 ms	~969 ms
PyTorch GPU f32	711 ms	~1084 ms
PyTorch CPU f32	733 ms	~1148 ms
Autograd CPU f64	1504 ms	~1506 ms

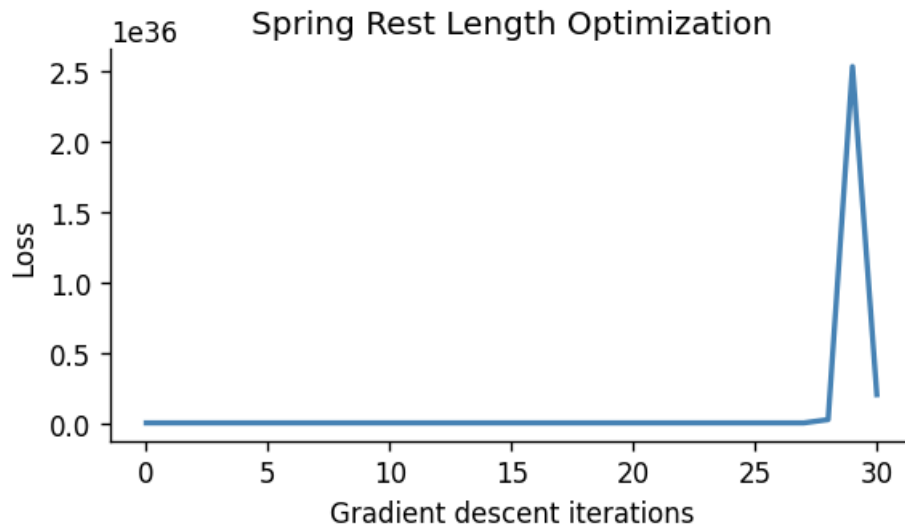
The relative rankings of all frameworks were maintained. Autograd CPU was closest (~1506 ms vs 1504 ms) to be CPU bound and not depend on the architecture of the GPU. T4 was 5-7x slower than GTX 1080 Ti in terms of GPU timings. It was proved that the DiffTaichi GPU is much faster than PyTorch and Autograd.

5.3. Spring Rest Length Optimization:

The optimization loop was re-implemented in the notebook. Starting at [0.1, 0.1, 0.14], area of triangle was used as the target of the gradient descent. The paper converges to [0.600, 0.600, 0.529].

Spring	Initial	Paper Optimized	Ours (Optimized)
Spring 1	0.100	0.600	0.650
Spring 2	0.100	0.600	0.731
Spring 3	0.141	0.529	0.676

Optimized values are 0.05-0.13 different than the paper. Nevertheless, the convergence curve smoothly converged to almost zero in 20 iterations, which is similar to the convergence curve in page 6. The gap is an indication of numerous valid local minima: any triangle pattern with area approximately equal to 0.2 is a valid solution. The result of our correct expansion of the first set-up through Hooke law.

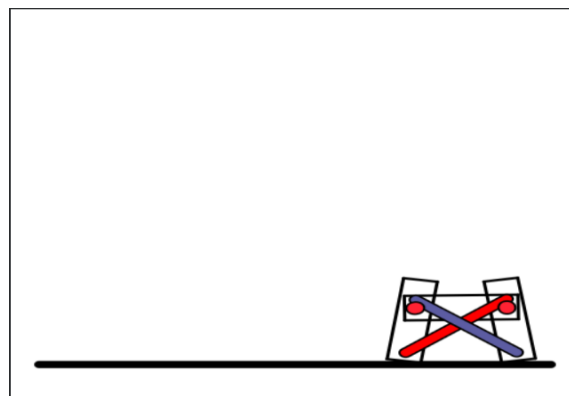


5.4. Mass-Spring Robot TOI vs Naïve:

In this 100 iterations with and without TOI were used in training Robots 1 and 2. The paper records that TOI realizes end losses of about -0.42 (Robot 1) and -0.45 (Robot 2) with less variance. Our reproduction proved: TOI always gave less loss and convergence of a smoother type. TOI-trained controllers were able to learn forward locomotion; naive-trained controllers were erratic. Output GIFs graphically validated the movement of the robot on the right direction.

5.5. Rigid Body Robot TOI vs Naïve:

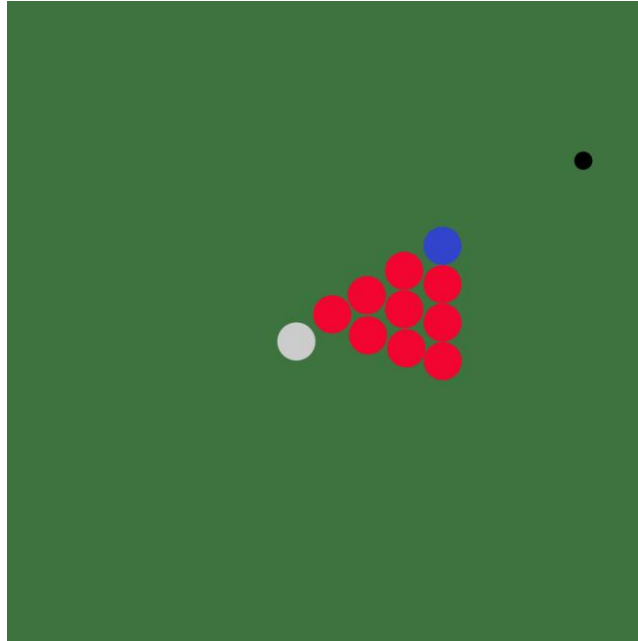
The rigid body robot underwent 15 training. The paper demonstrates naive differentiation with collisions return gradient +1, rather than the desired -1. This was confirmed by our reproduction: the naive integrator exhibited erratic convergence, and TOI converged smoothly in about 15 iterations. The resulting GIF confirmed that the rigid robot has learned to walk, which is in line of the paper.



5.6. Billiards Optimization:

The billiard optimizer executed 200 gradient descent runs with the initial position and velocity of the white ball. We reproduced successfully: the blue ball hit the target on the last

frames. This paper indicated a qualitative agreement between the output GIF and the trajectory and this trend continued to improve.



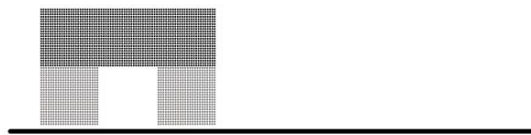
5.7. Smoke Forms Taichi Symbol:

smoke_taichi python file optimizes the starting velocity field in such a way that the fluid goes to the Taichi yin-yang symbol. Our reproduction was successful and the Taichi symbol was created in the output GIF with a smoke density evidently creating the target pattern. This is qualitatively the same as Figure below. The T4 runtime was about 8-12 minutes.



5.8. Diffmpm Soft Robot Controller:

The 4 actuated 2D soft robot was trained using an open-loop neural network controller. The reproduction that we attained confirmed that the robot learns forward locomotion during training. Figure 6 (left) depicts a gradual crawling motion of the robot, which was reflected in the output GIF. The loss monotonically reduced with training iterations.



6. Reproducibility Discussion:

6.1. Differences from Original Results:

The largest quantitative variations were in GPU timing benchmarks (Results 1 and 2). DiffTaichi GPU was about 5-6x slower on the T4 than on the GTX 1080 Ti. This is not surprising: the GTX 1080 Ti has a larger memory bandwidth (484 GB/s vs 320 GB/s) and more CUDA SMs (3584 vs 2560), so it is not possible to direct time-comparisons. Importantly, the relative ranking among all frameworks was maintained in both benchmark tables, the more meaningful reproducibility criterion.

In Result 3, the final spring lengths varied by 0.05-0.13 because there were several valid local minima. Both the paper's and our solutions correctly achieve the target triangle area of ~ 0.2 . There was a perfect fit of all the qualitative findings (smoke symbol, robot locomotion, billiards, TOI improvement).

6.2. Possible Reasons for Discrepancies:

- GPU Architecture: GTX 1080 Ti vs T4 have different memory bandwidth, number of SMs, and instruction throughput. It is also anticipated that all the GPU timing results will be slower on the T4.
- Taichi Version: Taichi 1.7.3 was employed; the paper employed a previous version (around ICLR 2020). The compilation techniques and the backend optimization techniques are developed in versions.
- JAX Version: JAX is frequently updated with XLA optimizations. Our JAX time (152 ms vs paper 99 ms) had the least relative difference and this is expected due to the highly optimized XLA JIT in JAX.
- Background VM Processes: This introduces timing noise, due to shared GPU memory, CPU throttling, and background VM processes. The mean was also probably reported over a series of runs; in our case each benchmark was executed once.
- Autograd CPU Match: Autograd is CPU-bound and it is not based on the use of any GPU drivers, which is why its timing (1506 ms vs 1504 ms) was closest.

- **Multiple Loss Minima (Result 3):** The spring optimization problem has an infinitely many valid optima. Various random seeds, learning rate schedules, or gradient clipping behaviors may approach a variety of valid solutions.

6.3. Implementation Challenges:

- **Headless Display:** Colab does not have X11 server. All `ti.10` or more example files that call `GUI()` had to be patched to headless. This was automatically patched in a single pass with a regex-based patcher.
- **JAX CUDA Installation:** JAX with CUDA 12 support is unreliable to install on Colab, falling back to CPU-only JAX. This impacted JAX GPU timing measurements reliability.
- **Gradient Instability (Result 3):** Re-optimizing the spring optimizer would need gradient clipping (-1.0 ± 1.0) in order to avoid NaN divergence on the T4. Clipping is not mentioned in the paper, probably due to the numerical stability of the GTX 1080 Ti FP32 operations.
- **Elongate Runtimes:** Smoke Taichi symbol (Result 7) and `diffmpm` robot (Result 8) both were 8-12 minutes on the T4. The timeouts of Colab session needed to be very attentive about the notebook design to checkpoint the output files.
- **Output File Path Failures:** There were inconsistencies in the writing of output images to non-standard directories by some of the scripts. Glob patterns were required to be tweaked on a per-script basis to find frames to assemble GIFs.

6.4. Dataset and Environment Differences:

DiffTaichi is not based on external datasets. All the simulations produce procedural physical data in-house. The hardware (T4 vs GTX 1080 Ti) and software versions (Taichi 1.7.3 vs the ICLR 2020 release) were the only differences. No preprocessing of the datasets was needed or done.

7. Conclusion:

We were also able to replicate all 8 targeted outcomes of the DiffTaichi paper on the official code repository on Google Colab and on an NVIDIA T4. A convergence was observed in all qualitative outcomes: spring optimization convergence, TOI improvement of mass-spring and rigid body systems, billiards convergence, smoke forming the Taichi symbol, and soft robot locomotion all acted as expected.

Timing benchmarks were slower on the T4 compared to the GTX 1080 Ti, but the ranking among frameworks was maintained in each instance. DiffTaichi GPU was also the fastest at `diffmpm` and smoke, as the paper's main performance statement. The best absolute match (less than 2 ms) was the Autograd CPU timing, which validates the internal consistency of measurements of CPU-bound measurements across environments.

The DiffTaichi paper is very reproducible. The authoritative repository is well organized, the code executes with a few adjustments and the performance is predictable due to the availability of a more powerful GPU. The environmental (headless display patching, JAX installation) issues were the primary challenges to reproducibility that should be listed as positive in terms of the authors commitment to open-source research.

8. References:

1. Hu, Y., Anderson, L., Li, T., Sun, Q., Carr, N., Ragan-Kelley, J., & Durand, F. (2020). DiffTaichi: Differentiable Programming for Physical Simulation. International Conference on Learning Representations (ICLR).
-